

Vaulted

EVERYTHING YOU OWN. PROTECTED. ORGANIZED. YOURS.

Security Posture Report

Platform Security Controls & Architecture

59

CONTROLS ACTIVE

0

CRITICAL OPEN

3

ENCRYPTED DBS

20

PENTEST PHASES

Classification: Confidential — Audit Use Only

Prepared for: Security Auditors & Compliance Review

Date: June 5, 2026 | **Version:** 1.0

AES-256-GCM

TLS 1.3

TOTP MFA

RBAC

JWT Rotation

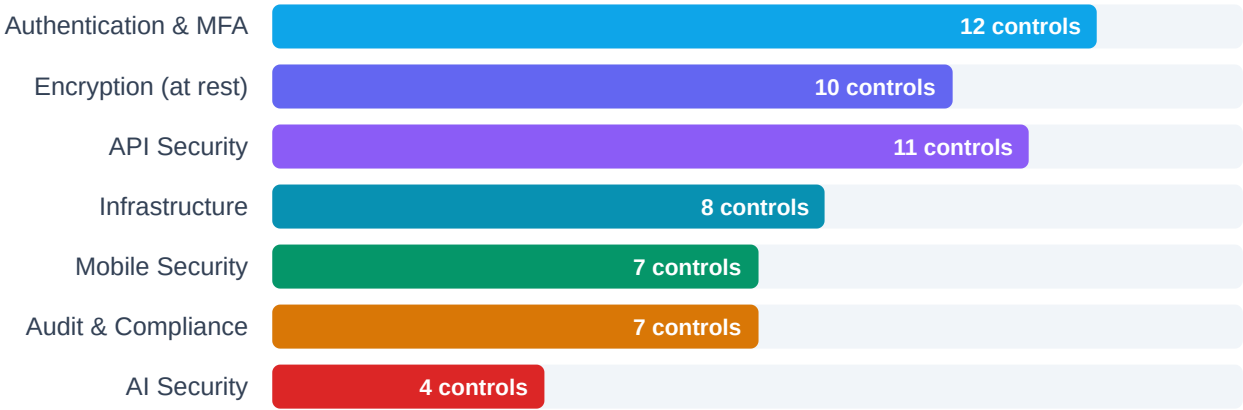
SOC 2 Ready

Table of Contents

1	Executive Summary
2	System Architecture Overview
3	Authentication & Access Control
4	Data Encryption at Rest & in Transit
5	API Security & Infrastructure Hardening
6	Mobile Security & AI Security
7	Audit Logging & Compliance
8	Penetration Test Results
A	Diagram: System Architecture
B	Diagram: Authentication & Token Flow
C	Diagram: Field-Level Encryption (FLE) Chain

Security Coverage by Domain

Active Controls per Security Domain



1. Executive Summary

Vaulted is a premium home inventory management platform for high-net-worth families in the USA. All sensitive data is encrypted at rest using AES-256-GCM with field-level isolation per tenant and per user. Access is governed by a multi-layer RBAC system with mandatory MFA for privileged roles. The platform has undergone 8 rounds of security hardening across all layers of the stack, resulting in **59 active security controls** with **zero critical or high-severity open findings**.

59
ACTIVE
CONTROLS

0
CRITICAL OPEN

0
HIGH OPEN

8
AUDIT ROUNDS

20
PENTEST
PHASES

100%
PENTEST PASS RATE

Security Hardening Summary by Round

Round	Focus Area	Controls Added	Severity Addressed
R-1	Authentication & Token Security	10	Critical High
R-2	RBAC & Multi-tenancy Isolation	8	Critical High
R-3	Field-Level Encryption (FLE)	9	High Medium
R-4	API Security & Rate Limiting	7	High Medium
R-5	Infrastructure & Container Hardening	8	Medium
R-6	Mobile Security Controls	7	Medium
R-7	AI Module Security	4	Medium
R-8	Audit Log Immutability & Compliance	6	Medium Low

Compliance Posture

- ✓ AES-256-GCM field-level encryption

✓ TLS 1.3 enforced end-to-end

✓ JWT with short expiry + rotation

✓ MFA mandatory for privileged roles

✓ Immutable audit log (2-year retention)

✓ RBAC with property-level scoping
- ✓ Certificate pinning on mobile clients

✓ Container hardening (no root, read-only FS)

✓ CSRF protection + secure cookies

✓ Token revocation via Redis blacklist

✓ Multi-tenant data isolation

✓ Penetration tested (20 phases, 100% pass)

2. System Architecture Overview

Vaulted uses a layered architecture with Cloudflare WAF and DDoS protection at the perimeter, a Caddy reverse proxy for TLS termination, and a NestJS API behind Docker containers. All databases are separate managed services with network isolation. No database is directly reachable from the internet.

Infrastructure Stack

Layer	Technology	Security Feature	Status
Perimeter	Cloudflare	WAF, DDoS protection, IP hiding	Active
TLS Termination	Caddy + Let's Encrypt	TLS 1.3, auto-renewal, HSTS	Active
API Runtime	NestJS (Docker, non-root)	Read-only FS, CAP_DROP ALL, PID limit	Active
Primary DB	MongoDB Atlas M0	FLE on 7 fields, TLS, no public IP	Active
Secondary DB	PostgreSQL Neon.tech	FLE on 6 fields, TLS, row-level RBAC	Active
Session Cache	Redis Upstash (TLS)	JWT blacklist, rate limiting, MFA TOTP lock	Active
Media Storage	Docker volume / GCP Storage	EXIF-stripped before AI processing	Active
Mobile Client	Flutter (iOS, Android, Web)	Cert pinning, Secure Enclave, jailbreak detection	Active

Network Security Zones

Zone 1 (Public): Cloudflare proxy → ports 80, 443 only. All other ports firewalled at GCP level.

Zone 2 (DMZ): Caddy container → terminates TLS, forwards to API on internal Docker network.

Zone 3 (Private): NestJS API → connects to MongoDB Atlas, PostgreSQL, Redis only via TLS. Databases not reachable from public internet (pending IP allowlist enforcement).

Docker Container Hardening

Control	Configuration
No root process	<code>USER node</code> in Dockerfile · <code>user: node:node</code> in Compose
Read-only filesystem	<code>read_only: true</code> + <code>tmpfs: /tmp</code>

Control	Configuration
No capability escalation	<code>cap_drop: [ALL] + no-new-privileges: true</code>
PID limit	<code>pids_limit: 100</code>
Image pinned	Base image locked to SHA-256 digest in Dockerfile
Secrets via env	<code>.env.prod</code> never committed to git

3. Authentication & Access Control

Authentication uses JWT Access Tokens (24h, in-memory) paired with httpOnly Refresh Tokens (7d, Secure cookie). MFA via TOTP is mandatory for Owner and Manager roles. All session states are tracked in Redis for real-time revocation. A 5-layer guard chain on every request enforces authentication, MFA verification, role authorization, and token revocation simultaneously.

JWT Token Architecture

Token Type	Expiry	Storage	Revocation	Claims
Access Token	24 hours	In-memory (Flutter)	Redis blacklist blacklist:{jti}	sub, role, tenantId, mfaVerified, typ:access
Refresh Token	7 days	httpOnly Secure cookie / Flutter SecureStorage	Redis session session:{uid}:{jti}	sub, jti, typ:refresh
Media Token	15 min	Request header	TTL expiry only	sub, mediaAccess, typ:media

Authentication Guard Chain (per request)



MFA Implementation

Control	Detail
Algorithm	TOTP (RFC 6238) — SHA-1, 30s window, 6-digit code
Required roles	Owner, Manager (mandatory). Staff, Auditor, Guest: optional.
Anti-replay	Redis key mfa:used:{userId}:{code} with 180s TTL — each TOTP code usable once
Rate limit	5 attempts/min per user (Redis sliding window)
Setup step-up	MFA enrollment requires current password re-verification
Secret storage	AES-256-GCM encrypted with per-user HKDF key (isolated per userId)

Role-Based Access Control (RBAC)

Role	Scope	MFA	Financial Data	User Mgmt
Owner	All properties	Required	Full	Full
Manager	Assigned properties	Required	No valuations	Invite only
Staff	Assigned rooms	Optional	None	None
Auditor	Assigned categories	Optional	Read-only	None
Guest	Specific items only	Optional	None	None

Token Refresh & Rotation Security

- ✓ Atomic Redis Lua script — prevents TOCTOU race condition on token rotation
- ✓ Refresh token replay detected → full session invalidation for that user
- ✓ New JTI issued on every refresh — old token immediately blacklisted
- ✓ Token type confusion prevented — `typ` claim validated at guard level
- ✓ Separate signing secrets for access, refresh, and media tokens

4. Data Encryption at Rest & in Transit

All sensitive fields in MongoDB and PostgreSQL are encrypted at the field level using AES-256-GCM with a 128-bit authentication tag. Keys are derived per-tenant (for inventory data) and per-user (for MFA secrets) using HKDF-SHA-256 from a scrypt-derived base key. No encryption key, IV, or plaintext value is ever logged or stored in a database column.

Encryption Key Derivation Chain

Step	Algorithm	Input	Output
1	scrypt (N=16384, r=8, p=1)	ENCRYPTION_KEY + ENCRYPTION_SALT (≥32 chars)	Base Key (256 bits, memory-only)
2a	HKDF-SHA-256	Base Key + info: vaulted-fle: {tenantId}	Tenant Key (256 bits, per-operation)
2b	HKDF-SHA-256	Base Key + info: vaulted-fle:{userId}	User Key (256 bits, per-userId)
3	AES-256-GCM	Tenant Key or User Key + random 12-byte IV	Ciphertext + 128-bit auth tag

Encrypted Fields Inventory

MongoDB · items collection (tenant key)

Field	Sensitivity
valuation.purchasePrice	Financial
valuation.currentValue	Financial
valuation.lastAppraisalDate	Sensitive
serialNumber	PII-adjacent
locationDetail	PII-adjacent

PostgreSQL · insurance_policies (tenant key)

Field	Sensitivity
provider	Sensitive
policyNumber	Confidential
totalCoverageAmount	Financial
premium	Financial

PostgreSQL · insured_items (tenant key)

Field	Sensitivity
coveredValue	Financial

PostgreSQL · users (per-user key)

Field	Sensitivity
mfaSecret	Critical — TOTP seed

Stored Ciphertext Format

```
{iv_hex}:{authTag_hex}:  
{ciphertext_hex}
```

Transit Encryption

- ✓ TLS 1.3 — client to Caddy
- ✓ TLS — Caddy to MongoDB Atlas
- ✓ TLS — Caddy to PostgreSQL (Neon)
- ✓ TLS (rediss://) — API to Redis (Upstash)

Field	Sensitivity
notes	Sensitive

5. API Security & Infrastructure Hardening

API Security Controls

Control	Implementation	Status
Rate Limiting — Login	5 failed attempts/min per IP, then 15-min logout (Redis)	Active
Rate Limiting — API global	100 req/min per tenant (NestJS ThrottlerGuard)	Active
Rate Limiting — AI Chat	20 req/min per tenant (atomic Redis SET NX EX)	Active
CSRF Protection	SameSite=Strict cookies + CSRF token on state-changing requests	Active
Input Validation	class-validator DTOs on all endpoints; ParseUUIDPipe on PostgreSQL IDs	Active
Error Sanitization	ObjectIds redacted to /:id in error responses; no stack traces	Active
IP Spoofing Prevention	trust proxy: 1 — only Caddy hop IP trusted, X-Forwarded-For not user-controlled	Active
Security Headers	Helmet.js: HSTS, CSP, X-Frame-Options, X-Content-Type	Active
Tenant Isolation	All queries scoped to tenantId from JWT — header injection ignored	Active
IDOR Prevention	MongoDB ObjectId ownership check on every item-level operation	Active
EXIF Stripping	sharp library strips metadata before image sent to AI processing	Active

Redis Session Key Architecture

Key Pattern	TTL	Purpose
session:{userId}:{jti}	7 days	Active refresh token tracking
blacklist:{jti}	7 days	Revoked access/refresh tokens
login:fail:{ip}	15 min	Login brute-force counter
mfa:attempts:{userId}	1 min	MFA rate limiting
mfa:used:{userId}:{code}	180 sec	TOTP anti-replay (one-time use)
ai:rate:{tenantId}	60 sec	AI chat rate limiting (atomic)
cache:dashboard:{tenantId}	5 min	KPI aggregation cache

Infrastructure Security Controls

Control	Detail	Status
Firewall	GCP: only ports 80, 443, one non-standard SSH port open	Active
SSH Hardening	Key-based only, password auth disabled, Fail2ban + UFW	Active

Control	Detail	Status
WAF / DDoS	Cloudflare proxy in front of all traffic — VM IP hidden	Active
Automated Backups	Daily GCP snapshots of the VM	Active
Secrets Management	.env.prod never committed; uploaded via gcloud scp	Active

6. Mobile Security & AI Security

Mobile Client Security Controls

Control	Platform	Implementation	Status
Certificate Pinning	iOS + Android	SHA-256 fingerprint via Dio interceptor (<code>package:crypto</code>)	Active
Secure Token Storage	iOS + Android	<code>flutter_secure_storage</code> → Keychain (iOS) / Keystore (Android)	Active
Access Token in Memory	All platforms	Access token never written to disk; only refresh token persisted	Active
Jailbreak / Root Detection	iOS + Android	<code>flutter_jailbreak_detection</code> — blocks app on compromised device	Active
Screenshot Guard	iOS + Android	Android <code>FLAG_SECURE</code> · iOS <code>SecureApplication</code> overlay	Active
Auto Token Refresh	All platforms	Dio interceptor retries on 401 with silent refresh	Active
Logout Cleanup	All platforms	<code>flutter_secure_storage</code> wiped on logout; access token nulled in memory	Active

AI Module Security Controls

Control	Detail	Status
AI Rate Limiting	Atomic <code>SET NX EX</code> in Redis — 20 req/min per tenant, no race condition	Active
EXIF Metadata Stripping	sharp strips GPS, camera model, and all EXIF data before image → Gemini	Active
AI Token Usage Logging	Every AI call logs tenant, model, tokens used to immutable audit log	Active
Tenant-Scoped RAG	Vector search (pgvector) filters by <code>tenantId</code> — cross-tenant data leakage prevented	Active

Certificate Pinning Rotation Procedure

Let's Encrypt certificates auto-renew via Caddy every ~90 days. To rotate the pinned certificate:

- ✓ Extract new SHA-256 fingerprint after Caddy renewal (`openssl s_client` command)
- ✓ Add new fingerprint to `AppConfig.pinnedCertFingerprints` (keep old + new during transition)
- ✓ Ship mobile release with both fingerprints — users with old app still connect
- ✓ After majority of users updated — remove old fingerprint in next release

7. Audit Logging & Compliance

Vaulted maintains an immutable audit log in PostgreSQL with 2-year retention. The `audit_logs` table is protected at the database level — UPDATE, DELETE, and TRUNCATE operations are revoked from the application user and enforced by before-triggers. Every authentication event, data access, configuration change, and AI usage is recorded with tenant context.

Audit Log Immutability Controls

Layer	Control	Detail
Application	Append-only AuditService	No update or delete methods exposed in the service
Database	Trigger: BEFORE UPDATE	Raises exception — prevents any row modification
Database	Trigger: BEFORE DELETE	Raises exception — prevents any row deletion
Database	REVOKE privileges	App DB user has no UPDATE, DELETE, or TRUNCATE on audit_logs
Retention	2-year policy	Records retained minimum 24 months per compliance target

Events Captured in Audit Log

Authentication Events

- ✓ Login success / failure (with IP)
- ✓ MFA verification success / failure
- ✓ Token refresh (with JTI)
- ✓ Logout (with session JTI)
- ✓ Token revocation events

Data Events

- ✓ Inventory item create / update / delete
- ✓ Insurance policy create / update
- ✓ User invite / role change
- ✓ Property create / modify

Security Events

- ✓ Rate limit triggered (login, MFA, API)
- ✓ IDOR attempt (cross-tenant access blocked)
- ✓ Invalid token / JWT validation failure
- ✓ MFA secret setup / change
- ✓ Session replay attack detected

AI Usage Events

- ✓ AI vision analysis (item, tenant, tokens)
- ✓ AI chat request (tenant, model, tokens)
- ✓ Insurance analysis request
- ✓ Maintenance risk scoring batch

Compliance Targets

Standard	Coverage	Status
SOC 2 Type II	Security, Availability, Confidentiality	In Progress
CCPA	Data subject rights, deletion, disclosure	In Progress

Standard	Coverage	Status
ISO 27001	Information security management	Post-Launch
OWASP Top 10 (2021)	All 10 categories tested and addressed	Covered

8. Penetration Test Results

Vaulted underwent a 20-phase automated penetration test covering all OWASP Top 10 categories plus additional domain-specific tests. All 20 phases passed. Tests are scripted and repeatable, running against the live production API at **api-vaulted.casacam.net**.

20
PHASES TOTAL

20
PASSED

0
FAILED

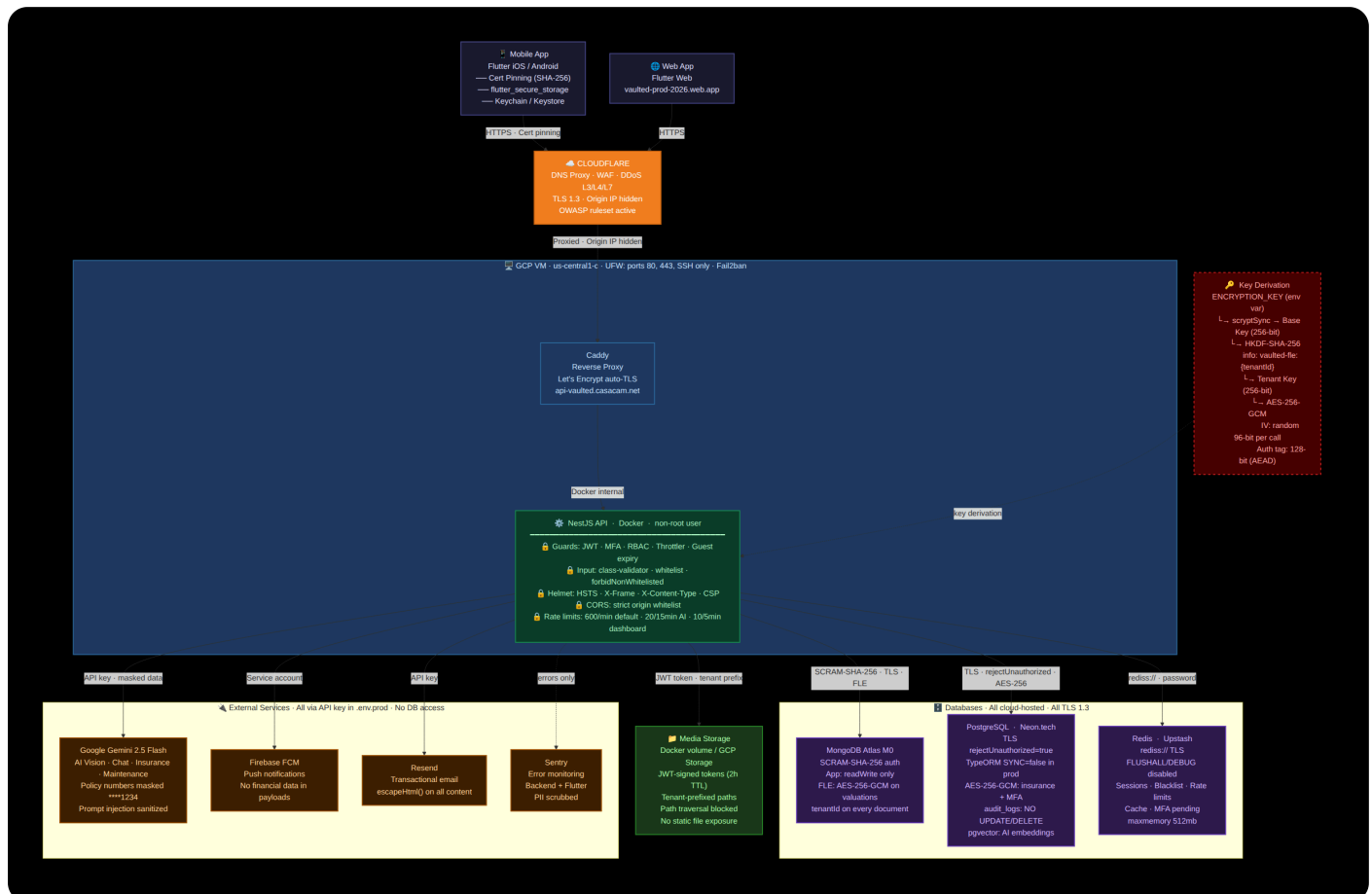
100%
PASS RATE

Penetration Test Phase Results

#	Phase	Category	Result
1	Authentication: login, invalid password, locked account	Auth	PASS
2	JWT validation: tampered payload, wrong secret, expired token	Auth	PASS
3	MFA: invalid code, replay attack, brute-force rate limit	Auth	PASS
4	Token refresh: rotation, replay detection, session invalidation	Auth	PASS
5	RBAC: role enforcement on all endpoints	AuthZ	PASS
6	IDOR: cross-tenant property and inventory access	AuthZ	PASS
7	Privilege escalation: self-role modification, invite bypass	AuthZ	PASS
8	SQL Injection: all PostgreSQL-backed endpoints	Injection	PASS
9	NoSQL Injection: MongoDB operator injection	Injection	PASS
10	XSS: stored and reflected in all text fields	Injection	PASS
11	Rate limiting: login brute-force, API flood, AI abuse	DoS Protection	PASS
12	Security headers: HSTS, CSP, X-Frame-Options, Referrer	Headers	PASS
13	Sensitive data exposure: error messages, stack traces	Info Disclosure	PASS
14	CSRF: cross-origin state-changing requests	CSRF	PASS
15	File upload: malicious content, path traversal	File Security	PASS
16	WebSocket: unauthenticated connections, cross-tenant events	WebSocket	PASS
17	Tenant isolation: X-Tenant-Id header injection	Multi-Tenancy	PASS
18	Token type confusion: access token used as refresh and vice versa	Auth	PASS
19	IP spoofing: X-Forwarded-For manipulation for rate limit bypass	API Security	PASS
20	Guest role expiry enforcement and role bypass attempts	AuthZ	PASS

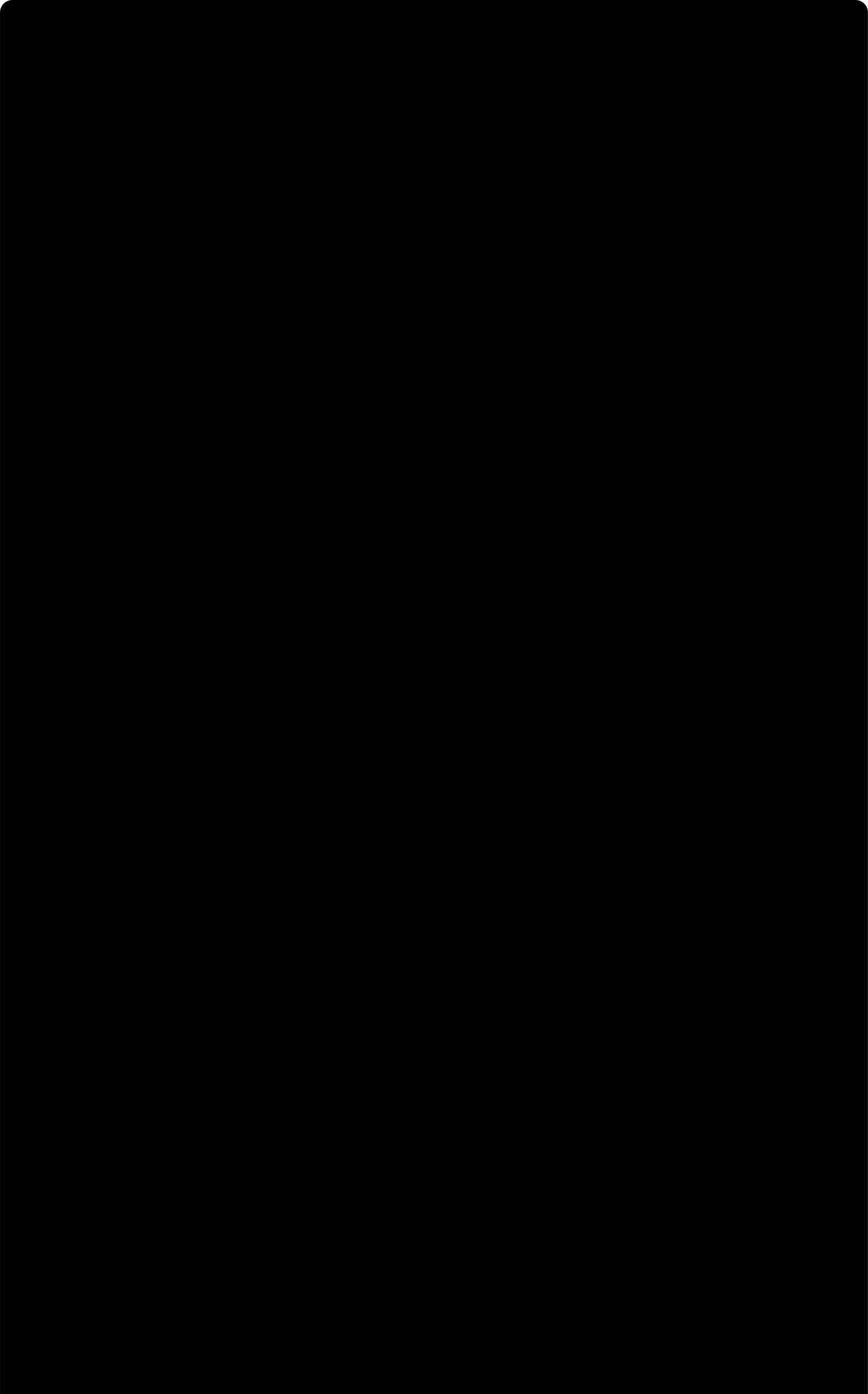
Appendix A: System Architecture Diagram

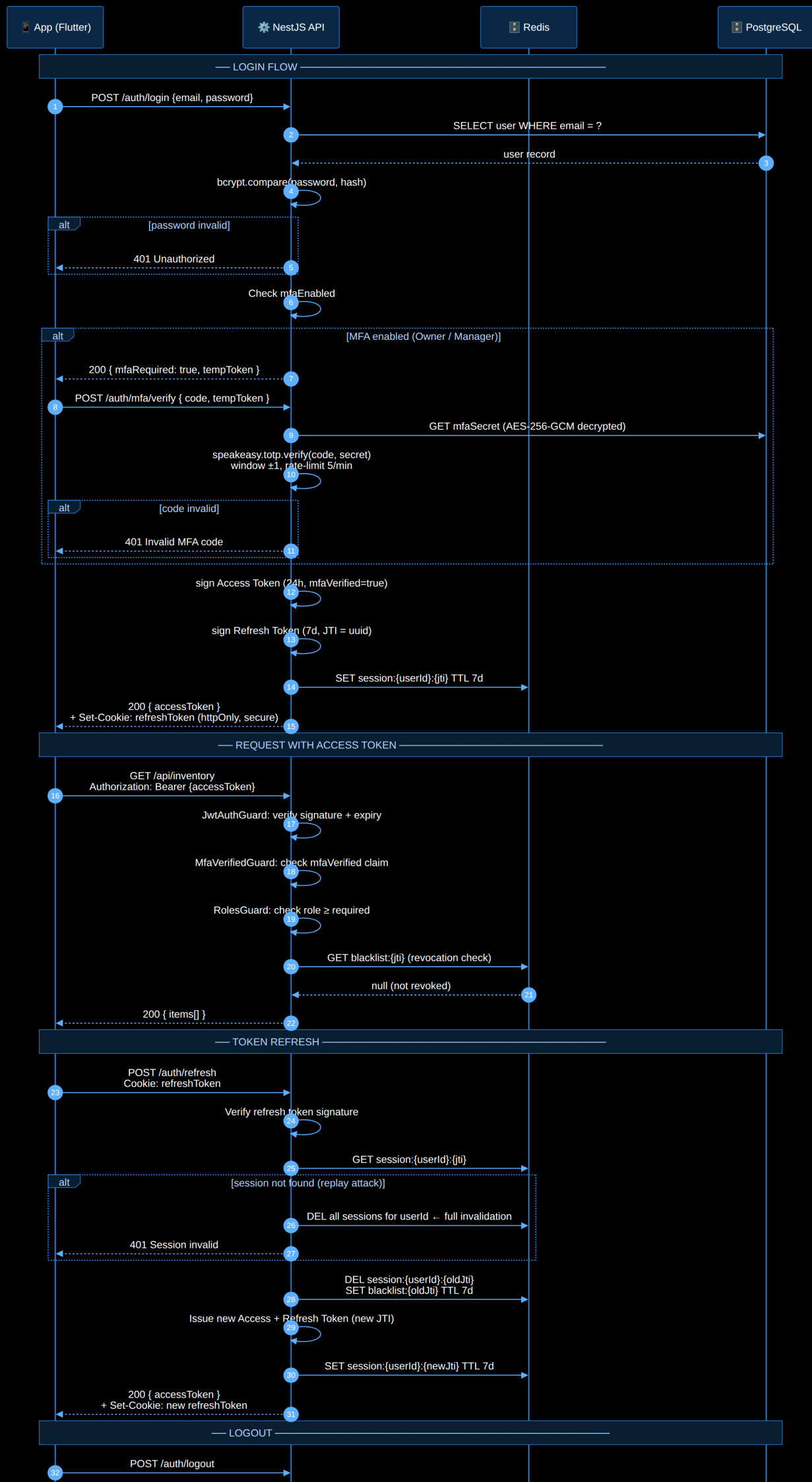
End-to-end system architecture showing all components: Cloudflare perimeter, Caddy TLS, NestJS API, MongoDB, PostgreSQL, Redis, Flutter clients, and GCP infrastructure. All external connections use TLS 1.3.

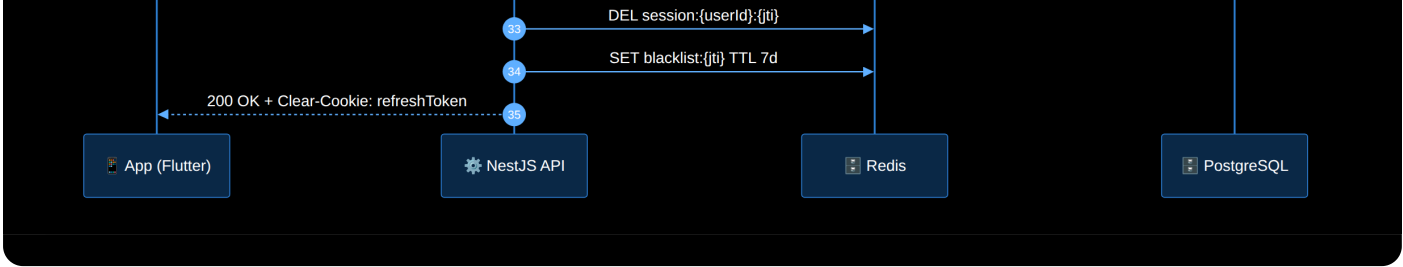


Appendix B: Authentication & Token Flow

JWT lifecycle: login → optional MFA verification → access token (24h, in-memory) + refresh token (7d, httpOnly cookie) → token rotation with Redis session tracking → logout with blacklisting. All refresh rotations use an atomic Lua script to prevent TOCTOU race conditions.







Appendix C: Field-Level Encryption (FLE) Chain

Key derivation from environment secrets through script to the Base Key, then HKDF-SHA-256 per tenant (for inventory/insurance fields) and per user (for MFA secrets). Each AES-256-GCM encryption uses a fresh random 12-byte IV. Ciphertext format: `{iv_hex}:{authTag_hex}:{ciphertext_hex}`.

